Docs-as-Code Proof-of-Concept

Tamás Dezső

May 29, 2026

Contents

List of Figures

List of Tables

1 Introduction

This document validates the `tdock` isolated technical documentation build environment. It tests standard markdown rendering, syntax highlighting for various languages, and the automatic vectorization of diagrams.

For the capabilities of the underlying Markdown engine, see the official **Pandoc:** The Universal Document Converter documentation. Pandoc Examples is also worth a check.

2 Styling

We support standard styling:

- **Bold text** for emphasis.
- *Italic text* for terminology.
- ***Bold-Italic*** for critical notes.
- `Monospaced code` for inline commands.

3 Tables

Tables are essential for documenting interface parameters or service specifications.

Interface	Protocol	Latency Target	Priority
Gx	Diameter	< 50ms	High
Ro	Diameter	< 100ms	Medium
SIP	UDP/TCP	< 30ms	High

Table 1: Service Interface Specifications

4 Mathematical Notation

For complex charging algorithms, we use standard LaTeX math support:

The transaction cost C is calculated as:

$$C = \sum_{i=1}^{n} (t_i \times r_i) + \text{tax}$$

5 Code-Blocks

By default, syntax highlighting uses the `tango` theme. The font should map to the Google/Ubuntu fonts requested in the YAML frontmatter.

5.1 C/C++

```c
#include <stdio.h>

typedef struct {
    int transaction_id;
    char status[16];
} Session;

int main() {
    Session s = {1042, "ACTIVE"};
    printf("Session %d is %s\n", s.transaction_id, s.status);
    return 0;
}
```

5.2 Lua

```lua
function check_node(elem)
  if elem.t == "CodeBlock" then
    return pandoc.RawBlock("html", "<!-- Code Block Exists -->")
  end
end
```

5.3 Bash

```bash
#!/bin/bash
echo "Starting compilation..."
mkdir -p build && cd build
cmake .. && make
```

5.4 Python

```python
def fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    elif n == 2:
        return [0, 1]
    else:
        seq = [0, 1]
        for i in range(2, n):
            seq.append(seq[-1] + seq[-2])
        return seq
```

6 Automated Diagrams

The blocks below are automatically parsed by the Pandoc diagram Lua filter, rendered via their respective engines, and embedded perfectly into the PDF using XeLaTeX.

On GitHub, Mermaid code-blocks are automatically rendered, and plantUML blocks can be visualized too with the PlantUML for GitHub Chrome Extension

6.1 PlantUML

- Website
- Language Reference Guide
- Style Guide

This is an example reference to Figure 1.

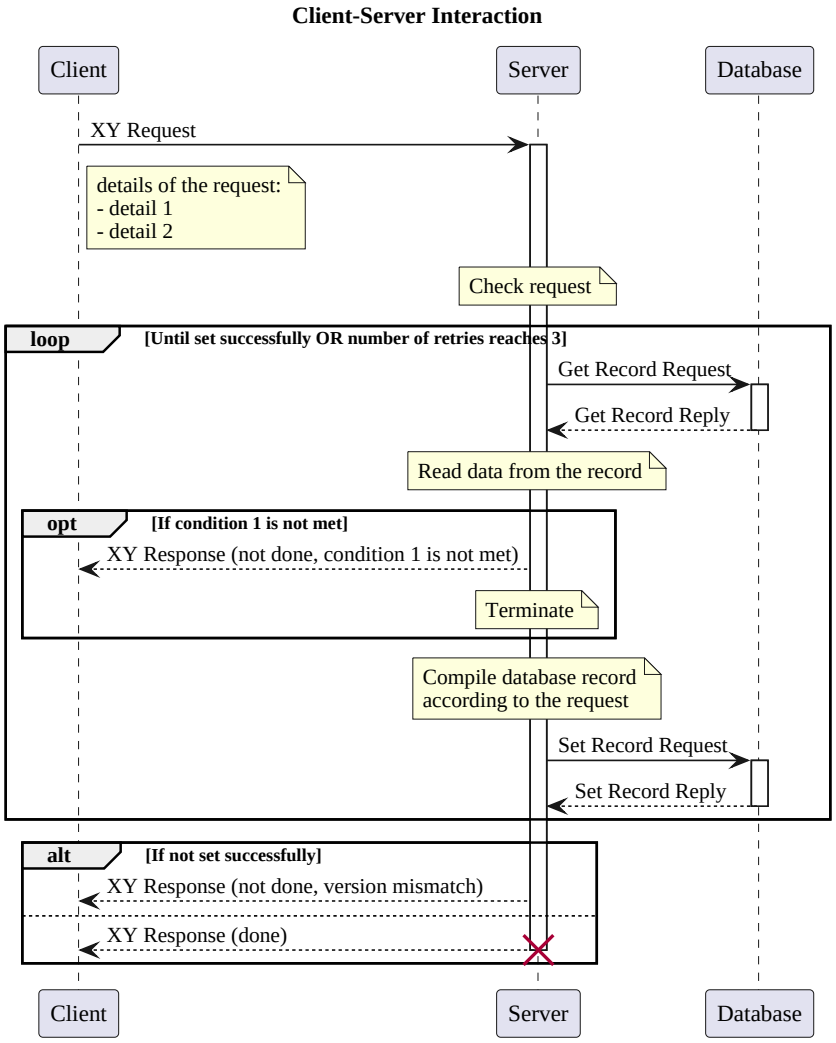

Figure 1: PlantUML Sequence Diagram Example

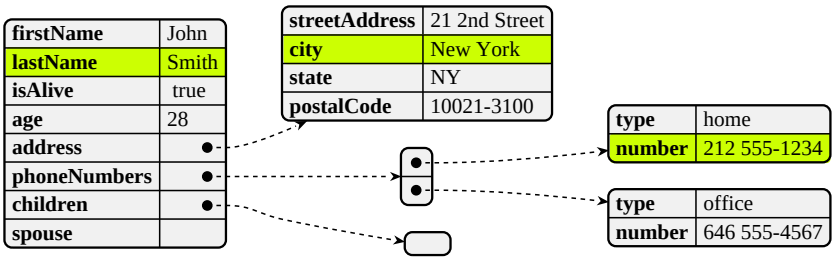

Figure 2: PlantUML Json Diagram Example

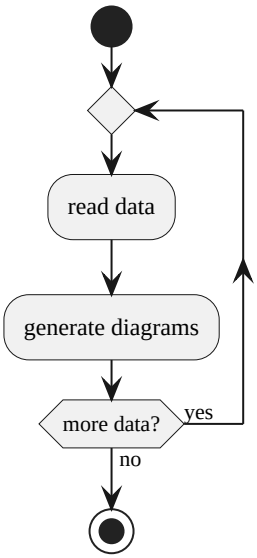

Figure 3: PlantUML Activity Diagram Example

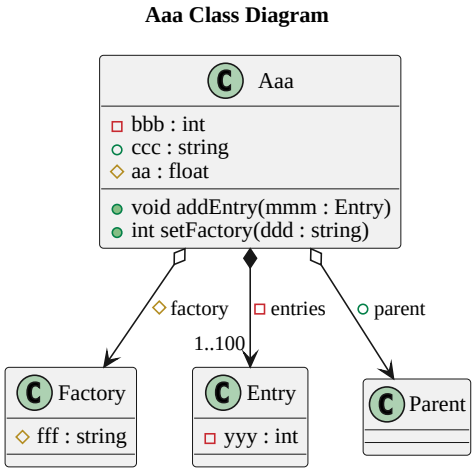

Figure 4: PlantUML Class Diagram Example

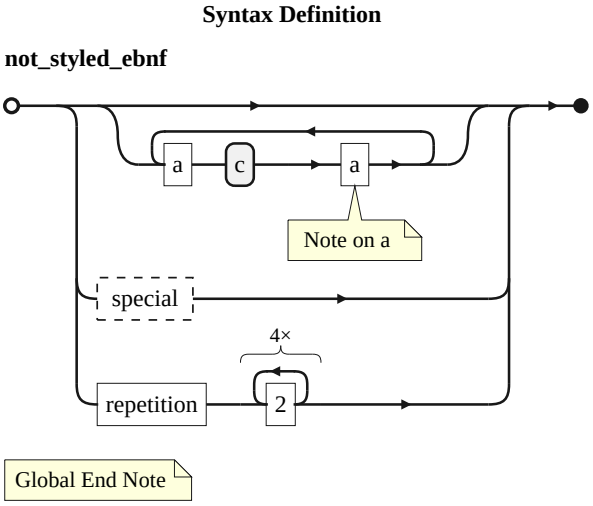

Figure 5: PlantUML EBNF Diagram Example

6.2 Mermaid

- Mermaid Open-Source Intro

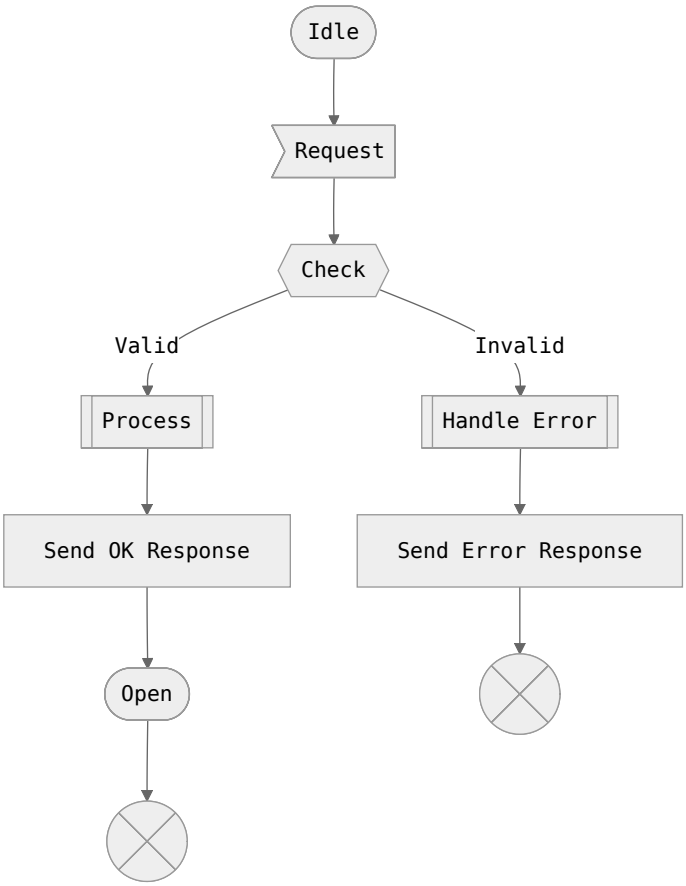

Figure 6: Mermaid Flowchart Example

6.3 Graphviz/DOT

- DOT Language Specification
- Gallery

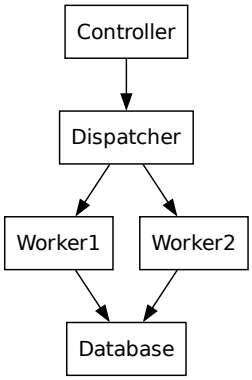

Figure 7: Graphviz/DOT Diagram Example